

Introduction Over the past few years, generative AI has started to become a normal part of software development. Tools that can generate code from simple prompts are now being used by students, indie developers, and even large companies working on commercial software. On the surface, this seems like a huge improvement—developers can work faster and solve problems more efficiently. Because of this, it’s not surprising that more teams are starting to rely on AI-generated code in real-world applications. However, just because something is useful doesn’t automatically mean it’s ethical, especially when that code is being shipped in commercial products.

One of the biggest issues isn’t just the existence of AI-generated code, but how it’s being used. In many cases, developers might not fully understand the code that the AI produces, yet still include it in finished products. This creates a situation where responsibility becomes unclear—if something breaks or introduces a security issue, it’s harder to know who is accountable. At the same time, there’s also the problem of transparency. If developers don’t acknowledge when or how AI was used, it raises concerns about authorship. These problems become more serious in commercial settings, where software affects real users.

There are also broader concerns about where AI-generated code comes from. Since these systems are trained on large amounts of existing code, including open-source projects, it’s possible that they reproduce patterns or structures without proper acknowledgement to the original authors.

This paper argues that using generative AI to generate code for commercial applications is only ethical under certain conditions. Specifically, developers need to understand the code they use and be transparent about how AI contributed to the final product.

Background and Stakeholders To understand the ethical issues around AI-generated code, it helps to first clarify what is being discussed and who is affected by it. In this context, software engineers can range from individual developers working on small personal projects to large teams building complex commercial systems. Generative AI refers to tools that can produce code based on natural language prompts, often trained on large datasets that include publicly available code and documentation. Commercial applications do not just mean software that is sold directly—they can also include internal tools or services that companies rely on to operate.

The ethical concern arises because AI-generated code exists in a kind of gray area. It is not fully written by the developer, but it is also not clearly attributable to any single original source. Research has shown that generative AI systems can reproduce patterns from training data, raising concerns about copyright, ownership, and proper attribution [2], [7]. These systems are also known to produce inaccurate or misleading outputs, sometimes referred to as “hallucinations,” which can make their use risky in real-world applications [3], [5].

Several key stakeholders are affected by this issue. The most immediate group is software engineers. These tools can significantly improve productivity and reduce the time required to write code, but they also introduce risks if developers rely on outputs they do not fully understand. Studies on AI-assisted writing and development emphasize that users often overtrust generated content, which can lead to errors being overlooked [3]. This becomes especially problematic when code is deployed in production environments without careful review.

Companies and employers are another major stakeholder. From a business perspective, AI tools offer clear advantages in terms of efficiency. However, they also create potential legal and ethical risks. For example, if AI-generated code incorporates copyrighted material without proper attribution, companies may face liability issues. Concerns about the relationship between AI systems and open-source code have already been raised in industry discussions, particularly regarding whether AI tools unfairly benefit from publicly available work [9].

End users are also affected, even if indirectly. They depend on software to function correctly and securely. If developers deploy AI-generated code that contains bugs, users may experience failures, data breaches, etc. The broader impact of unreliable or misleading AI-generated content has already been observed in other domains, where large-scale use has created challenges for institutions trying to maintain quality and trust [10].

Another important stakeholder group is the open-source community. Many AI systems are trained on publicly available repositories, which often include contributions from developers who did not explicitly consent to their work being used in this way. This raises ethical concerns about fairness and respect for intellectual labor, especially when the resulting outputs are used in commercial products [7], [9].

Finally, the organizations that develop generative AI systems also play a significant role. They are responsible for how models are trained and what data is included.. Ethical guidelines for AI emphasize the importance of transparency, accountability, and fairness in system design, suggesting that developers of these tools share responsibility for how they are ultimately used [4], [8].

Overall, the use of AI-generated code in commercial applications affects a wide range of stakeholders, each with different interests and risks. Because of this, the ethical question is not just about whether the technology works, but whether it is being used responsibly and respectfully.

Current State of Gen AI Generative AI has quickly become a common tool in software development. Many developers now use AI tools regularly for tasks like writing boilerplate code and debugging. In fact, recent studies show that a large majority of developers use generative AI on a daily basis, and that these tools can significantly reduce the time required for routine

programming tasks [6]. Because of this, AI is increasingly being integrated into real-world development workflows, including commercial applications.

One of the biggest advantages of generative AI is the productivity boost it provides. Developers can generate functional code much faster than writing everything from scratch, especially for simple or repetitive tasks. In many cases, AI acts as a kind of assistant that helps fill in gaps in knowledge or speeds up development. Research suggests that these tools are particularly effective for tasks like documentation, testing, and basic implementation work [6]. Overall, this has made software development more accessible and efficient.

However, these benefits come with significant limitations. One major issue is that AI-generated code is not always correct. Studies have shown that AI-generated programs can contain logical errors, incorrect outputs, or even fail to compile in some cases [5]. Even when the code looks correct, it may include subtle bugs or poor design choices that are difficult to detect. This is partly because generative AI does not truly understand the code it produces—it is simply predicting patterns based on training data. As a result, it can generate outputs that seem convincing but are actually flawed [3].

Security is another major concern. Research has found that AI-generated code can contain more issues compared to human-written code, including misconfigurations and unsafe practices [6]. In real-world usage, many developers do not consistently review or audit this code before using it, which increases the risk of security problems. Recent reports show that while AI-generated code is widely adopted, oversight and validation have not kept up, creating potential risks for companies deploying this code in production environments ([IT Pro](#)).

Another important issue is over-reliance. Even though many developers do not fully trust AI-generated code, a large number still use it without thoroughly checking it first ([TechRadar](#)). This creates a situation where code is being shipped without a full understanding of how it works. Over time, this can also lead to reduced skill development, since developers may rely on AI instead of learning how to solve problems themselves [6].

Finally, the current state of generative AI reflects a broader pattern: the technology is powerful, but still imperfect. While it is very effective for simple tasks, it often struggles with more complex, multi-step problems or large-scale systems. As a result, human oversight is still necessary, even as AI becomes more integrated into development workflows. The gap between how widely these tools are used and how well they are understood or managed remains one of the biggest challenges moving forward.

Technical Background To understand why AI-generated code can be risky, it helps to have a basic idea of how these systems actually work. Most modern generative AI tools are based on large language models, which are trained on massive datasets that include text, documentation,

and publicly available code. During training, the model learns patterns in how words and code are structured, but it does not actually “understand” what it is generating in a human sense [6].

At a high level, these models work by breaking input text into smaller pieces called tokens and then predicting what token should come next based on probability. When a developer gives a prompt, the model processes it and generates code one token at a time, choosing the most likely sequence based on what it has seen during training. This means the output is essentially a statistical prediction rather than a logically reasoned solution [3]. Because of this, the model can produce code that looks correct and well-structured, even if it contains hidden errors.

One important implication of this process is that AI-generated code is heavily influenced by its training data. Since these models are trained on large collections of existing code, they may reproduce patterns, structures, or even specific implementations that resemble previously written code. This is part of what makes them effective, but it also creates concerns about copyright and originality, especially if the generated output is used in commercial products [7]. The model is not intentionally copying code, but it also has no way to ensure that its output is completely original.

Another limitation is that the model does not have true reasoning or understanding. It cannot verify whether the code it generates is correct, secure, or efficient. As a result, it may produce outputs that compile but behave incorrectly, or include subtle bugs that are difficult to detect. Research has shown that AI-generated content can include inaccuracies and “hallucinations,” where the model produces information that seems plausible but is actually wrong [5]. In the context of software development, this can lead to code that appears functional but fails under certain conditions.

Security is also a concern at the technical level. Because the model does not understand best practices in a meaningful way, it may generate code that includes unsafe patterns or vulnerabilities. For example, it might suggest insecure ways of handling user input or fail to properly validate data. These issues can become serious when the code is used in commercial applications, where security and reliability are critical [6].

Overall, the technical process behind generative AI explains many of the ethical concerns. Since the model is based on pattern prediction rather than true understanding, developers cannot assume that its outputs are correct or original. This is why human oversight is necessary, and why using AI-generated code without fully understanding it can lead to significant problems.

Ethical Analysis To evaluate whether it is ethical for software engineers to use generative AI to produce code for commercial applications, Social Contract Theory provides a useful framework. This theory is based on the idea that society functions through a set of implicit agreements between individuals. People agree to follow certain rules, such as honesty, fairness, and accountability—in exchange for trust and cooperation from others. In the context of software

engineering, there is an unspoken contract between developers, companies, users, and the broader programming community.

Under this framework, the ethical issue with AI-generated code is not just about whether it works, but whether its use violates these shared expectations. One of the most important parts of this social contract is trust. Users trust that software developers understand the code they produce and that it has been properly tested and reviewed. However, when developers rely on AI-generated code without fully understanding it, this trust is weakened. If something goes wrong—such as a bug or security vulnerability—it becomes unclear who is responsible. This lack of accountability directly conflicts with the expectation that developers stand behind their work.

Another key part of the social contract in software development is honesty, especially regarding authorship and contribution. Traditionally, developers are expected to write their own code or properly credit the sources they use. With generative AI, this boundary becomes less clear. If a developer uses AI-generated code but does not acknowledge it, they may be presenting the work as entirely their own, even though it was partially produced by a system trained on other people's contributions. This raises concerns about fairness, particularly for developers whose code may have been included in training datasets without explicit consent [7], [9]. From a social contract perspective, failing to acknowledge AI use can be seen as a violation of the expectation of transparency.

There is also an implicit agreement between developers and the open-source community. Open-source contributors share their work with the expectation that it will be used in ways that respect licensing terms and give proper credit. However, generative AI systems can produce code that resembles existing work without attribution, creating tension between convenience and fairness. This challenges the idea that participation in the software ecosystem is based on mutual respect and shared benefit. If AI systems allow companies to profit from this work without recognition, it may undermine the willingness of developers to contribute in the future.

At the same time, Social Contract Theory does not necessarily imply that the use of generative AI is always unethical. Instead, it suggests that the technology must be used in a way that maintains these shared agreements. For example, if developers take responsibility for reviewing and understanding AI-generated code, they are still upholding their role in the contract. Similarly, if companies are transparent about their use of AI and implement safeguards to prevent misuse, they are maintaining trust with users and other stakeholders. Ethical guidelines for AI emphasize transparency and accountability as key principles, reinforcing the idea that these expectations are part of a broader social agreement [4], [8].

Ultimately, from a Social Contract perspective, the ethical acceptability of AI-generated code depends on whether its use respects the expectations that allow the software ecosystem to function. When developers fail to understand the code they use or do not acknowledge AI

involvement, they weaken trust, reduce accountability, and create unfair conditions for others. However, when AI is used responsibly—with proper oversight, transparency, and respect for existing contributions—it can fit within the existing social contract. This means that generative AI itself is not inherently unethical, but its misuse can easily violate the agreements that developers rely on.

Safeguards and Responsibilities Given the current state of generative AI, it's pretty clear that the technology itself isn't the main problem—it's how people are using it. Because of that, making AI-generated code ethical in commercial applications mostly comes down to putting the right safeguards in place and making sure the people involved take responsibility for how it's used.

One of the most important safeguards is simply requiring developers to review and understand any code generated by AI before using it. Since AI models can produce incorrect or misleading outputs, blindly trusting the results can lead to bugs, security issues, or poor design choices [3], [5]. This means developers should treat AI-generated code the same way they would treat code written by another person—something that needs to be checked and tested. In practice, this could involve code reviews, testing procedures, and making sure the developer can actually explain how the code works before it gets deployed.

Another key safeguard is transparency. Developers and companies should clearly acknowledge when AI tools have been used in the development process, especially in commercial projects. Without this, it becomes harder to assign responsibility or evaluate the originality of the work. Lack of transparency also raises ethical concerns about honesty and authorship, particularly if AI-generated code is presented as entirely human-written. Ethical guidelines for AI use emphasize the importance of transparency and accountability, suggesting that users should disclose when AI plays a role in producing content [4], [8].

There are also technical safeguards that could help reduce risks. For example, AI tools could include built-in systems to detect whether generated code closely matches existing copyrighted material. This would help address concerns about licensing and intellectual property, which are already a major issue in discussions about AI-generated code [7], [9]. Additionally, improving documentation or adding warnings when the model is uncertain could help developers better understand the limitations of the output they are using.

Responsibility is not limited to individual developers. Companies also have a role in setting standards for how AI tools are used within their organizations. This could include policies requiring code review for AI-generated content, training developers on the risks of using these tools, and ensuring that security checks are in place before deployment. Since companies ultimately benefit from the use of AI, they also need to take responsibility for any negative outcomes, including legal or ethical issues.

At the same time, the organizations that develop generative AI systems also share responsibility. They control how models are trained and what data is included, which directly affects the kind of outputs these systems produce. Ethical concerns about bias, copyright, and misuse suggest that AI developers should prioritize fairness, transparency, and accountability when designing their systems [4], [8]. This could include being more open about training data sources or implementing stronger safeguards against problematic outputs.

Overall, making the use of AI-generated code ethical is less about banning the technology and more about using it responsibly. Developers need to understand and verify the code they use, companies need to enforce clear policies, and AI developers need to design systems with ethical risks in mind. Without these safeguards, the issues of unclear responsibility and lack of transparency become much harder to justify, especially in commercial applications where the stakes are higher.

Conclusion At this point, it's pretty clear that generative AI is not going away anytime soon, and it's already becoming a normal part of software development. The ability to quickly generate code is genuinely useful, and in many cases it can make developers more productive and help lower the barrier to entry for learning programming. However, just because the technology is useful does not mean it is always being used in an ethical way, especially when it comes to commercial applications.

The biggest issue is not the technology itself, but how it is used. Throughout this paper, the main concerns have been lack of understanding and lack of transparency. When developers use AI-generated code without fully understanding how it works, they are effectively taking responsibility for something they did not completely create. This becomes a serious problem when that code is deployed in real products that affect users. At the same time, failing to acknowledge the use of AI raises questions about honesty, authorship, and whether the work is being represented accurately.

Looking at this through ethical frameworks, it becomes harder to justify careless use of AI-generated code. From a responsibility standpoint, developers still have a duty to ensure that their code is correct and safe, regardless of how it was created. From a broader perspective, widespread misuse of AI tools could reduce trust in software systems and create unfair situations for developers whose work may have been used without proper credit [2], [8]. These concerns suggest that ethical use of AI requires more than just technical ability—it also requires accountability and awareness.

That said, generative AI can still be used ethically if certain conditions are met. Developers need to take the time to understand and verify the code they use, companies need to enforce clear guidelines and review processes, and there needs to be more transparency about when AI tools are involved. Without these safeguards, the risks start to outweigh the benefits, especially in commercial contexts.

For me personally, this changes how I think about using AI tools in my own work. It's easy to rely on them for quick answers, but that doesn't mean I actually understand what I'm shipping. Going forward, I think the most important takeaway is that using AI responsibly means treating it as a tool, not a replacement for understanding. If developers can do that, then AI can be a helpful part of the development process. If not, it becomes a shortcut that creates more problems than it solves.

Bibliography

- [1] Daniel Schlagwein and Leslie Willcocks. 2023. *"ChatGPT et al.: The ethics of using (generative) artificial intelligence in research and science."* Information Technology & People.
- [2] Author(s). 2025. *"Ethical Issues in AI-Generated Texts: A Systematic Review."* International Journal of Human-Computer Interaction.
- [3] Author(s). 2025. *"Artificial intelligence-assisted academic writing: recommendations for ethical use."* Advances in Simulation.
- [4] Author(s). 2024. *"Publication Ethics in the Era of Artificial Intelligence."* PubMed Central (PMC).
- [5] Author(s). 2024. *"Practical Considerations and Ethical Implications of Using Artificial Intelligence in Writing Scientific Manuscripts."* PubMed Central (PMC).
- [6] Author(s). 2024. *"Research Integrity and GenAI: Ethical Challenges Across Research Phases."* arXiv.
- [7] Author(s). 2025. *"Defining Ethically Sourced Code Generation (ES-CodeGen)."* arXiv.
- [8] Author(s). 2021. *"Ethics of AI: A Systematic Literature Review of Principles and Challenges."* arXiv.
- [9] TechRadar. 2024. *"The AI paradox: can AI and open source development co-exist?"*
- [10] The Washington Post. 2026. *"AI-generated text is overwhelming institutions: setting off a no-win arms race with AI detectors."*